



**POLITECNICO
DI TORINO**

Dipartimento
di Automatica e Informatica

Gli Ordinamenti Ricorsivi

Paolo Camurati



I dati da ordinare

- I dati da ordinare non sono necessariamente sempre e solo interi.
- Generalizzazione: i dati da ordinare appartengono ad un tipo Item definito come struct
 - uno dei campi = *chiave* di ordinamento
 - restanti campi = dati aggiuntivi
- funzioni di interfaccia a oggetti di tipo Item
 - lettura/scrittura
 - generazione di valori casuali
 - accesso alla chiave
- operatori relazionali su oggetti di tipo Item

Item definito come:

- ADT di I classe
 - Quasi ADT
- quelli belli, fatti per bene
trattati più avanti
- Tipi di Dato Astratto
- Abstract
Data
Type

Tipologie: L'item può essere

1. semplice scalare e chiave coincidente un unico valore che coincide con la chiave
2. vettore dinamico di caratteri e chiave coincidente la chiave è il vettore di caratteri
3. scalare e vettore di caratteri sovradimensionato staticamente in una struct uno o più campi scalari/vettoriali dentro una struct con uno di essi usato come chiave
4. scalare e vettore di caratteri allocato dinamicamente in una struct.

Cfr. *Puntatori e strutture dati dinamiche* cap. 6.2

Esempio: Quasi ADT – tipologia 1

lavoriamo su un singolo carattere

```
...  
#define maxKey 100
```

definizione di un nuovo tipo Item

```
typedef int Item;
```

consideriamo il dato come INTERO ma lo chiamiamo ITEM

scan Item ITEMscan(); restituisce un item

equal (int ITEMeq(Item A, Item B);

NOT equal int ITEMneq(Item A, Item B);

Less Than int ITEMlt(Item A, Item B);

Greater Than int ITEMgt(Item A, Item B);

```
void ITEMshow(Item A);
```

```
Item ITEMrand();
```

NON entriamo più dentro l'ITEM (struct) per fare operazioni ma accederemo ai suoi campi tramite una funzione

prototipi di funzioni su dati di tipo Item

```
...  
int ITEMeq(Item A, Item B) {  
    return (A == B);  
}  
  
int ITEMneq(Item A, Item B) {  
    return (A != B);  
}  
  
int ITEMlt(Item A, Item B) {  
    return (A < B);  
}  
  
int ITEMgt(Item A, Item B) {  
    return (A > B);  
}  
...
```

ritorna intero usato come booleano
0 falso
!= vero (noi usiamo 1 di solito)

implementazione
di funzioni su dati
di tipo Item

```
Item ITEMscan(){
    Item A;
    printf("item = "); scanf("%d", &A);
    return A;      l'item letto viene ritornato
}

void ITEMshow(Item A) {
    printf("%6d \n", A);
}

Item ITEMrand() {
    Item A= maxKey*((float)rand()/RAND_MAX);
    return A;
}
```

#usiamo rand() per generare un numero casuale
#lo rapportiamo a RAND_MAX per ottenere un numero compreso tra 0 e 1
#lo moltiplichiamo per il massimo numero accettabile



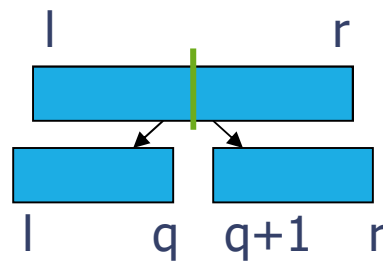
problema gran scan?
con elastico come racchiudo tutti i chiodi inseriti nel legno

ottengo poligono per connettere 2 pt
qualsiasi a lunghezza minima
contornando il poligono

Merge Sort (von Neumann, 1945)

Divisione:

- due sottovettori SX e DX rispetto al centro del vettore.



Ricorsione

- condizione di terminazione: con 1 ($l=r$) o 0 ($l>r$) elementi il vettore è ordinato
- merge sort su sottovettore SX
- merge sort su sottovettore DX

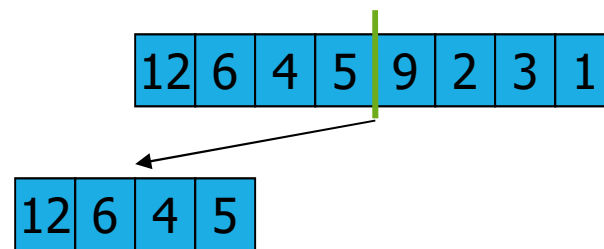
Ricombinazione:

- fonde i due sottovettori ordinati in un vettore ordinato.

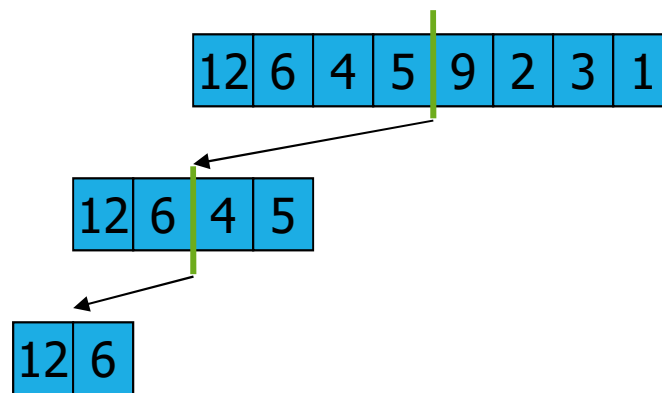
Esempio

12	6	4	5	9	2	3	1
----	---	---	---	---	---	---	---

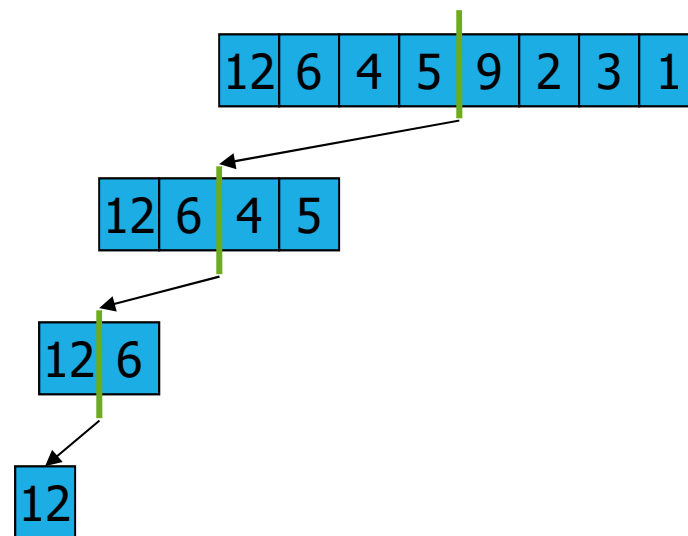
Divisione ricorsiva:



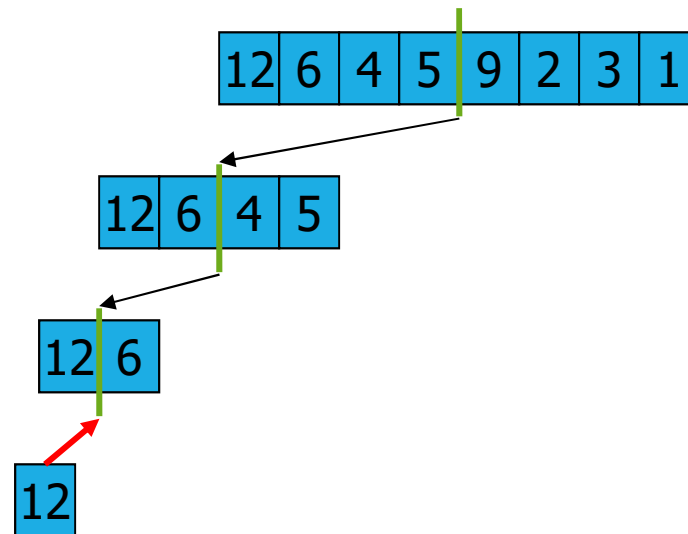
Divisione ricorsiva:



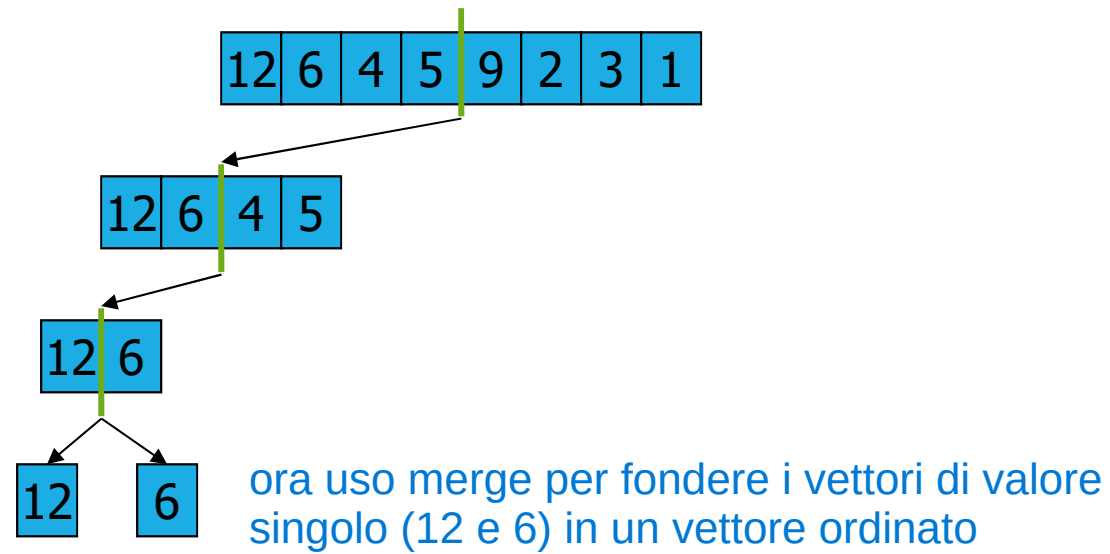
Divisione ricorsiva:



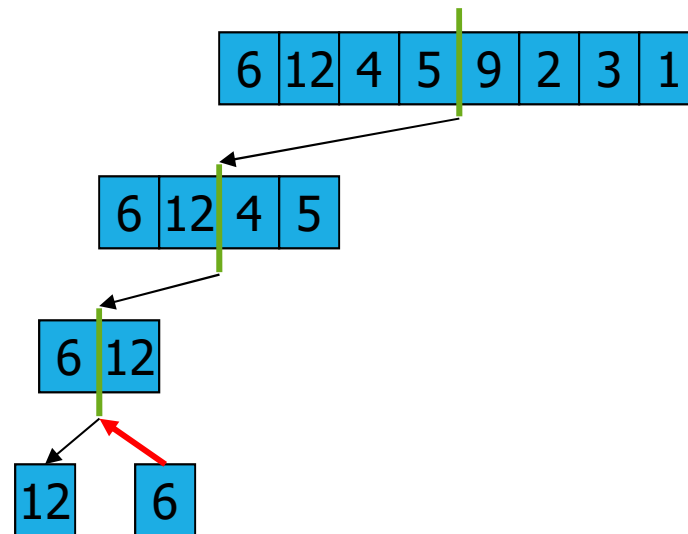
Ricombinazione:



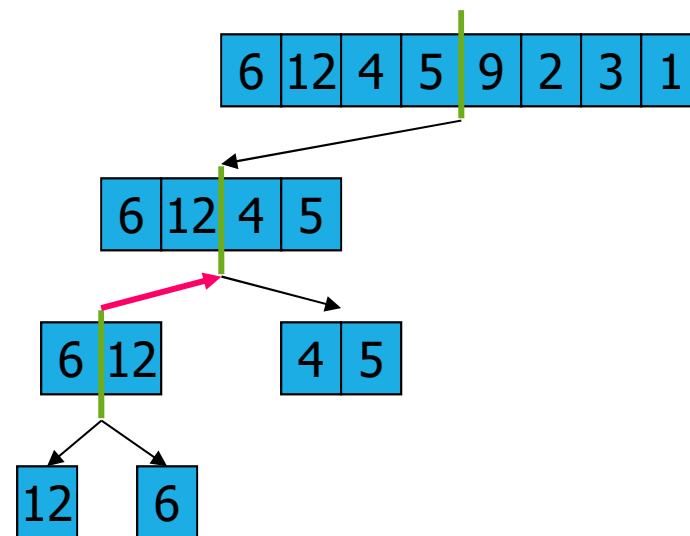
Divisione ricorsiva:



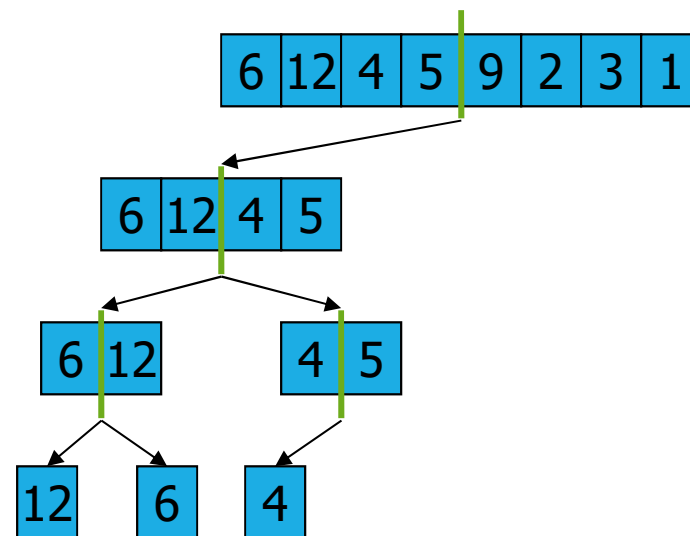
Ricombinazione:



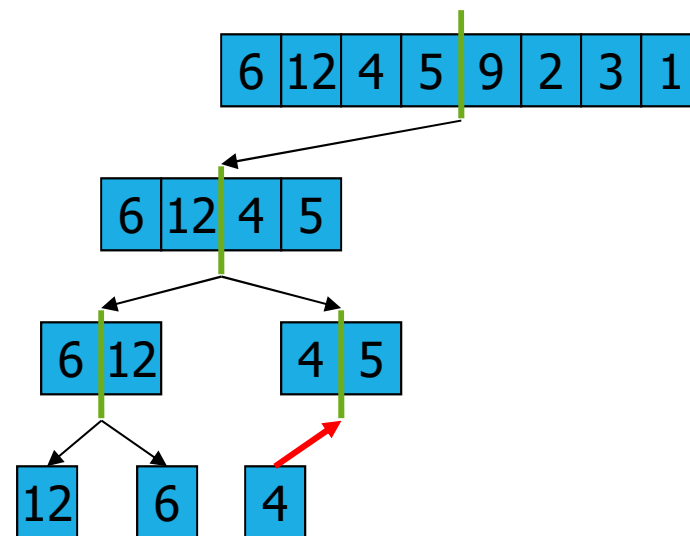
Divisione ricorsiva:



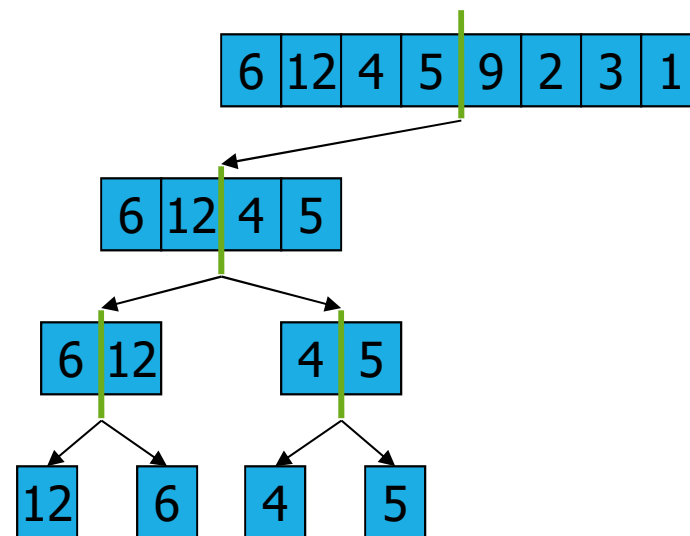
Divisione ricorsiva:



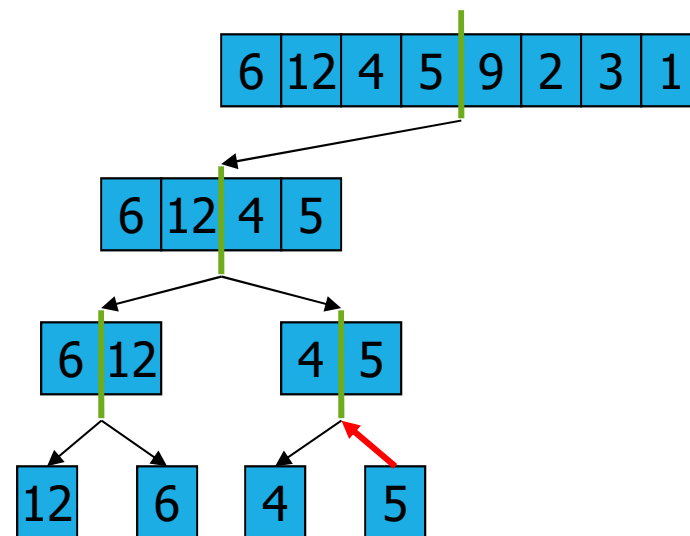
Ricombinazione:



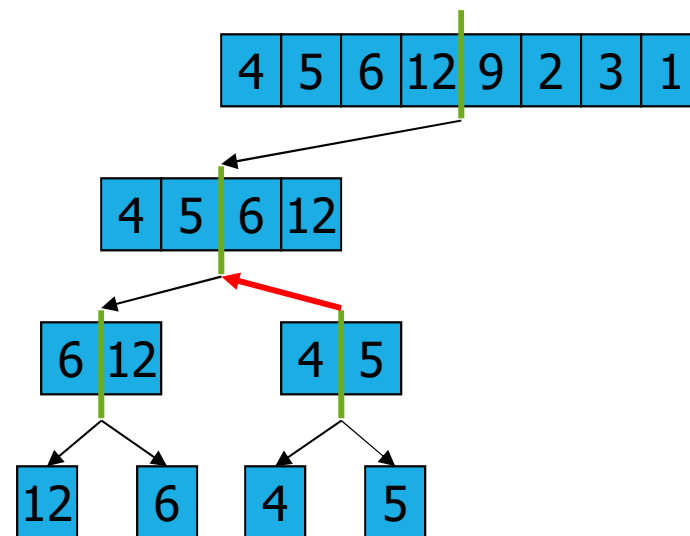
Divisione ricorsiva:



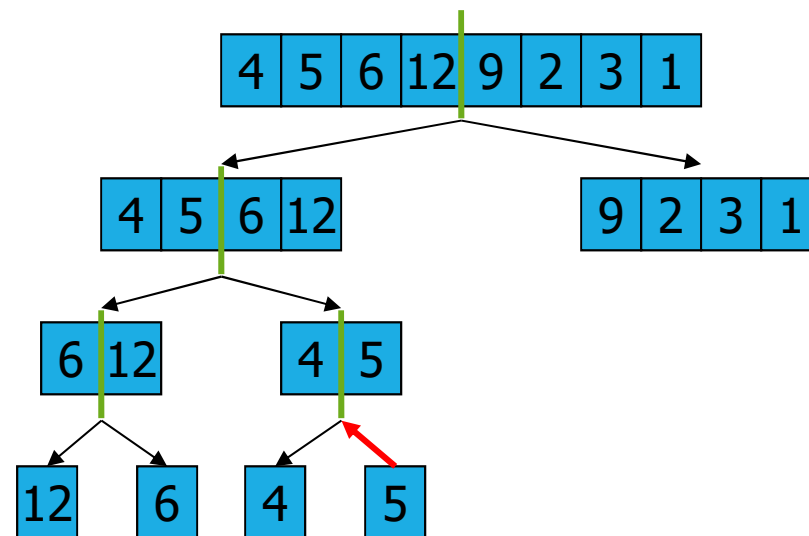
Ricombinazione:



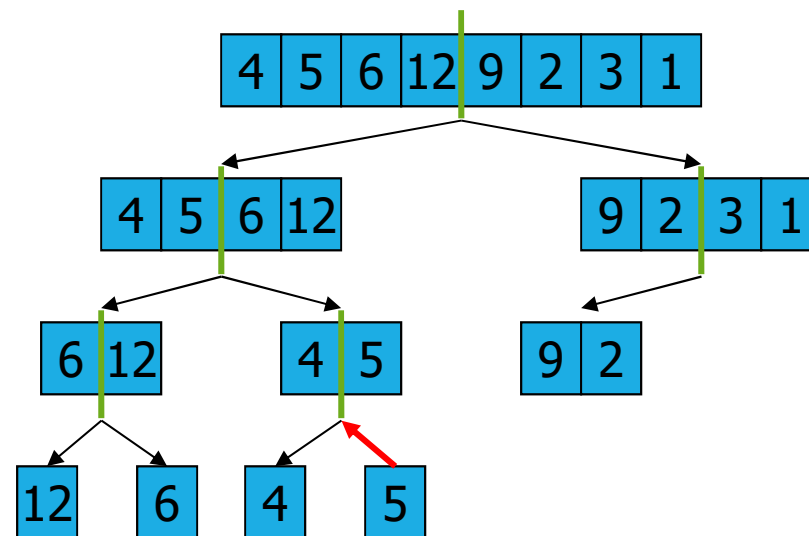
Ricombinazione:



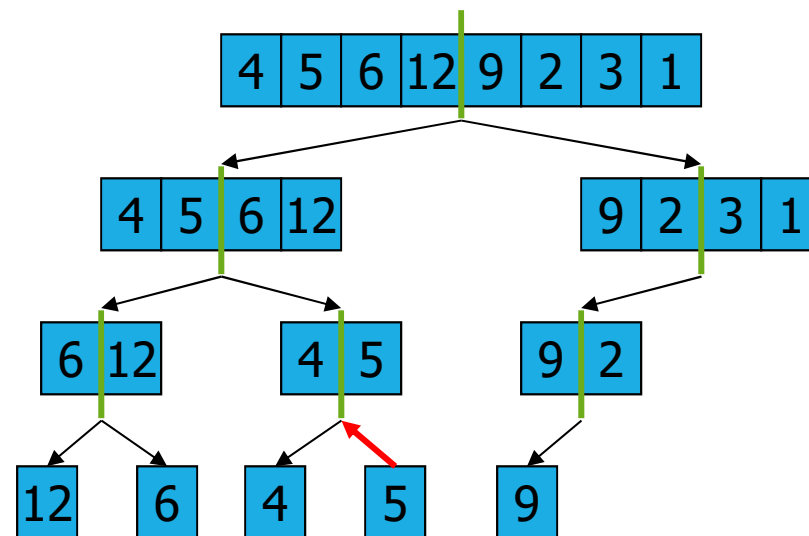
Discesa ricorsiva:



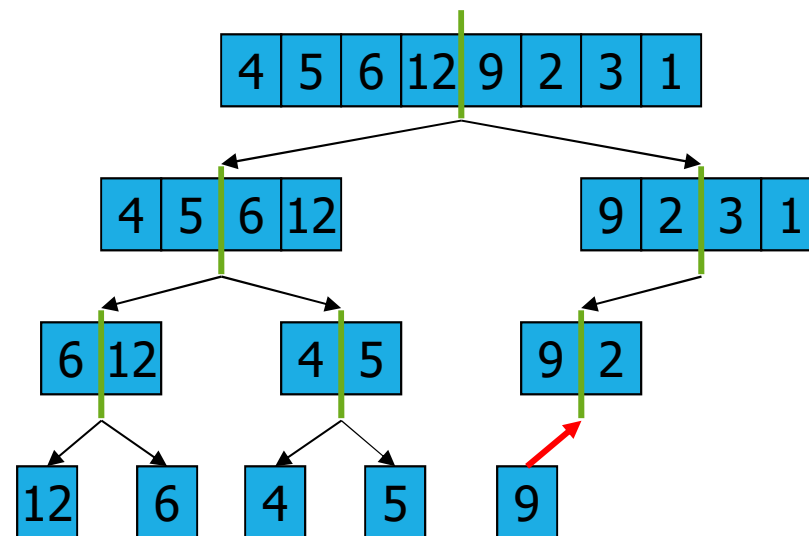
Discesa ricorsiva:



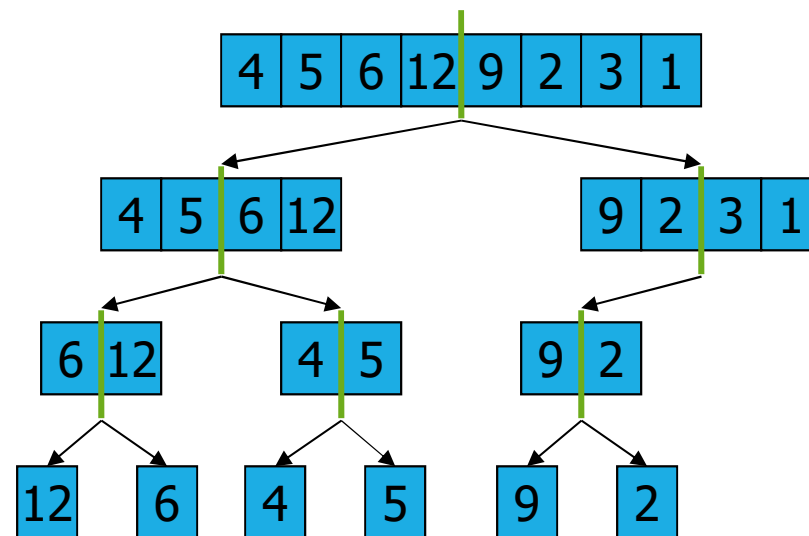
Discesa ricorsiva:



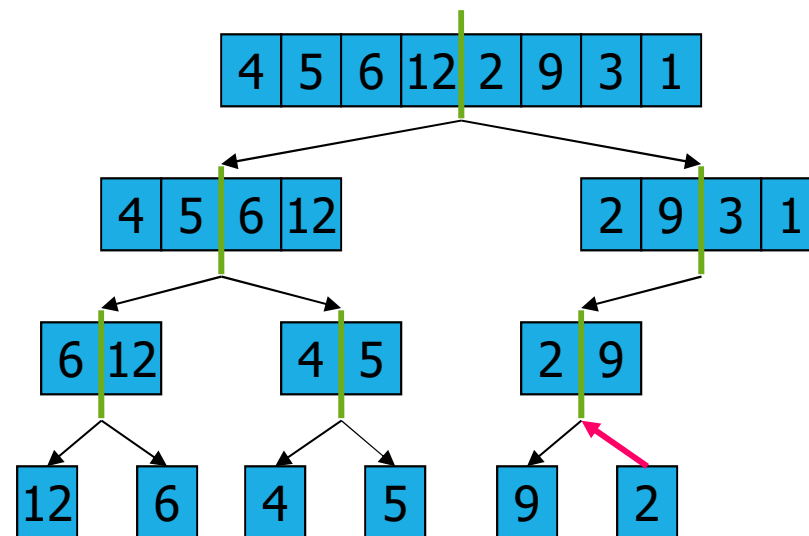
Ricombinazione:



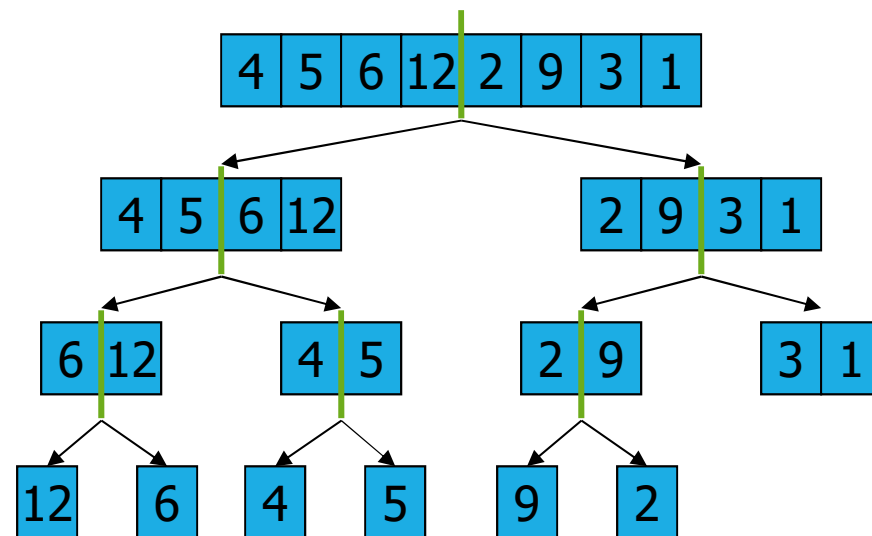
Discesa ricorsiva:



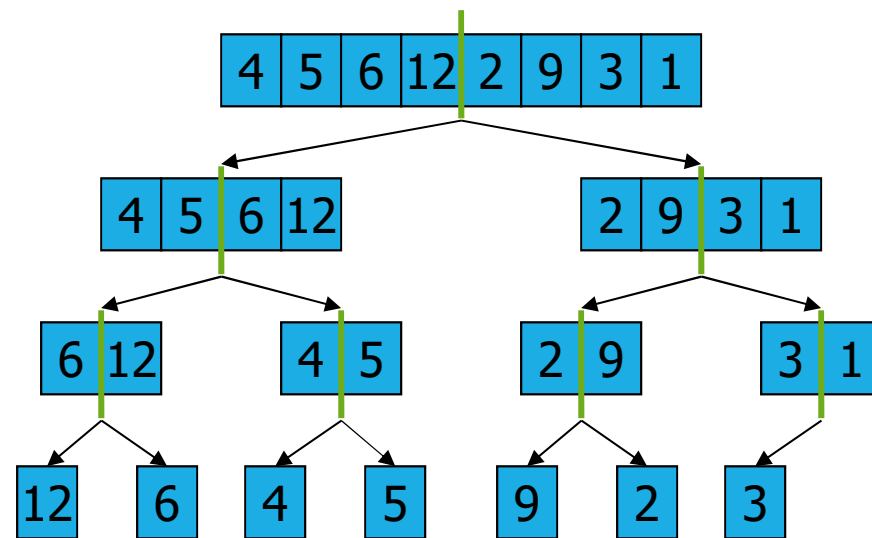
Ricombinazione:



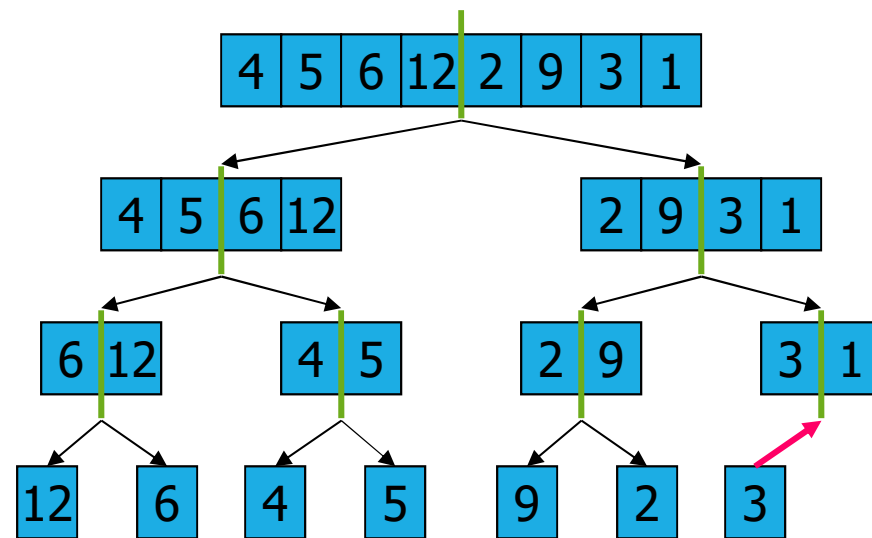
Discesa ricorsiva:



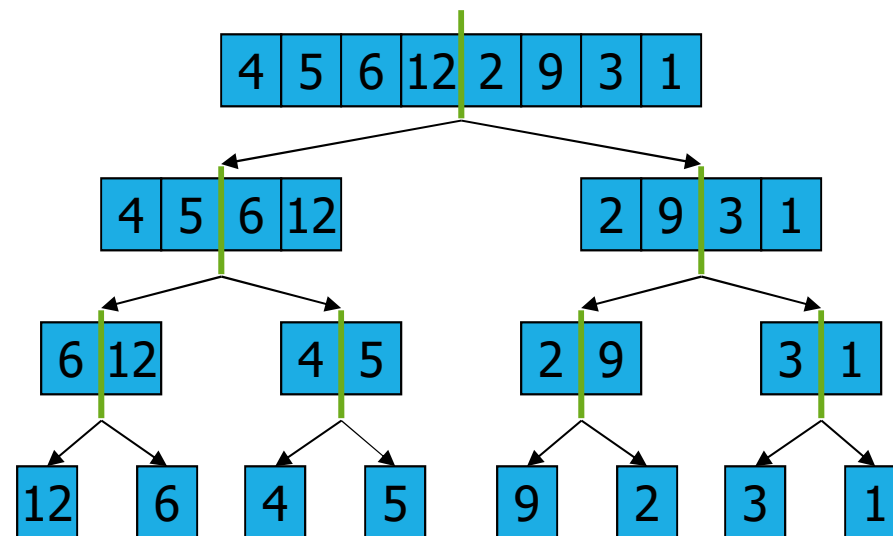
Discesa ricorsiva:



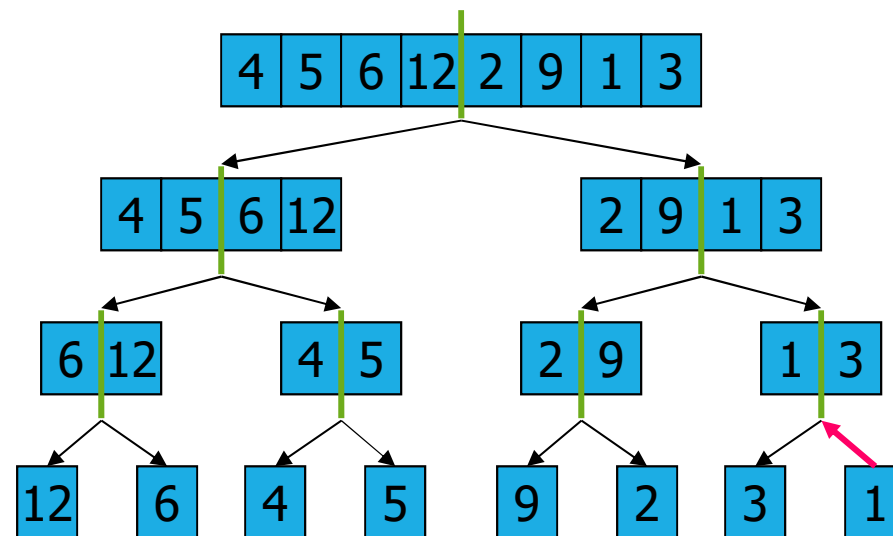
Ricombinazione:



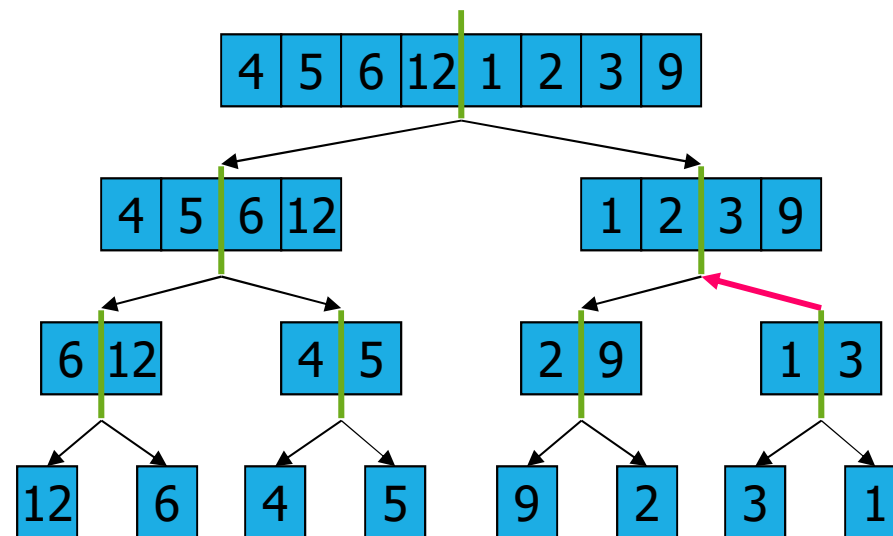
Discesa ricorsiva:



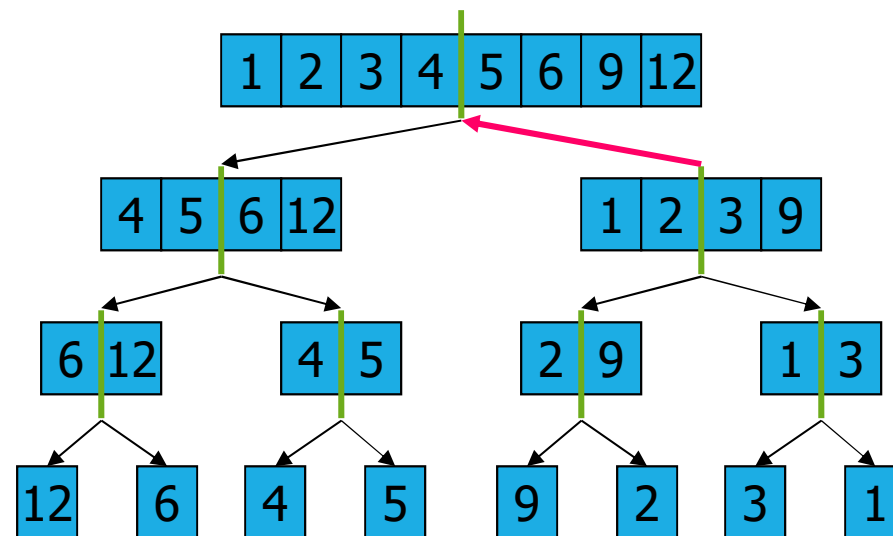
Ricombinazione:

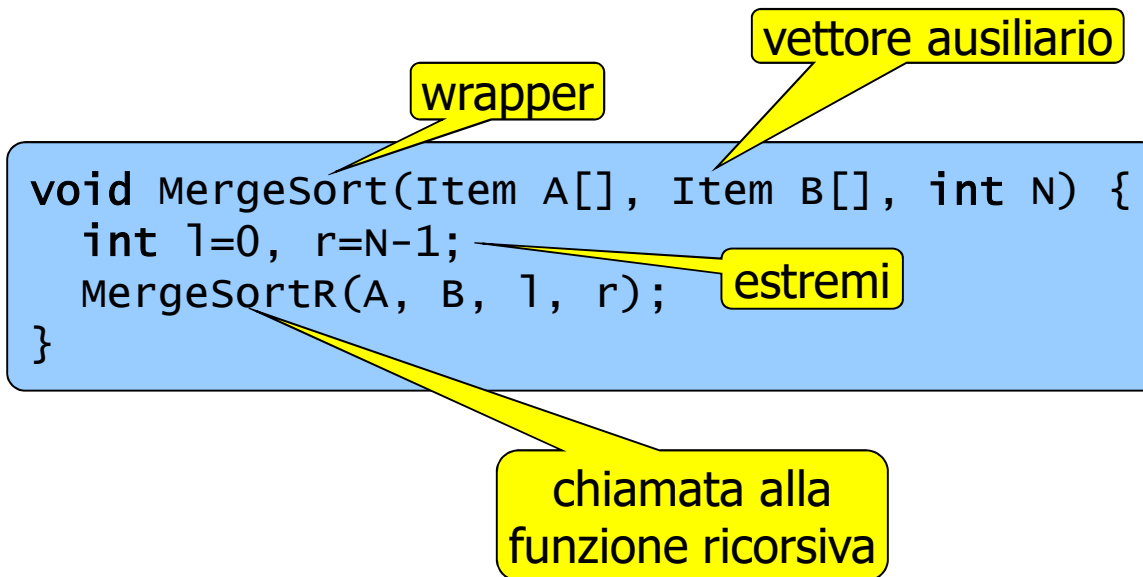


Ricombinazione:



Ricombinazione:



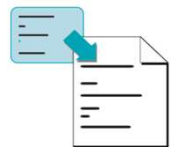


dove lo genero:

Il main alloca in maniera statica o dinamica il vettore di item ausiliario ed è il main a passarlo al merge sort

OPPURE

allocare dinamicamente item ausiliario DENTRO il merge ogni volta che serviva



RecursiveSort.c

```
void MergeSortR(Item A[], Item B[], int l, int r){  
    int q;  
    if (l >= r) terminazione  
        return; divisione  
    q = (l + r)/2;  
    MergeSortR(A, B, l, q); chiamata ricorsiva  
    MergeSortR(A, B, q+1, r); chiamata ricorsiva  
    Merge(A, B, l, q, r);  
}
```

ricombinazione

2-way Merge

- ipotesi: la dimensione del vettore A è una potenza di 2 $N = 2^p$
- fusione di 2 (2-way) sottovettori di A ordinati di uguale dimensione per ottenere un vettore ordinato di dimensione doppia
- generalizzabile a k vettori (k-way Merge)
- indice q per dividere sottovettori di A a metà in 2 sottovettori sinistro e destro
- sottovettore sinistro con indice $l \leq i \leq q$
- sottovettore destro con indice $q+1 \leq j \leq r$
- vettore ausiliario B di dimensione N con indice $l \leq k \leq r$ per memorizzare i risultati delle fusioni passato come parametro

Approccio:

- scorrere i sottovettori sinistro e destro mediante gli indici i e j e il vettore B mediante l'indice k
 - se è esaurito il sottovettore sinistro ($i > q$), ricopiare gli elementi rimanenti del sottovettore destro in B
 - altrimenti se è finito il sottovettore destro ($j > r$), ricopiare gli elementi rimanenti del sottovettore sinistro in B
 - altrimenti confrontare l'elemento corrente $A[i]$ del sottovettore sinistro con quello del sottovettore destro $A[j]$
 - se $A[i] \leq A[j]$, ricopiare $A[i]$ in B e avanzare i , j resta invariato
 - altrimenti ricopiare $A[j]$ in B e avanzare j , i resta invariato
- I confronti avvengono mediante operatori relazionali su Item.

```

void Merge(Item A[], Item B[], int l, int q, int r) {
    int i, j, k;
    i = l;
    j = q+1;
    for (k = l; k <= r; k++)
        if (i > q)
            B[k] = A[j++];
        else if (j > r)
            B[k] = A[i++];
        else if (ITEMlt(A[i], A[j]) || ITEMeq(A[i], A[j]))
            B[k] = A[i++];
        else
            B[k] = A[j++];
    for (k = l; k <= r; k++)
        A[k] = B[k];
    return;
}

```

esaurito sottovettore SX

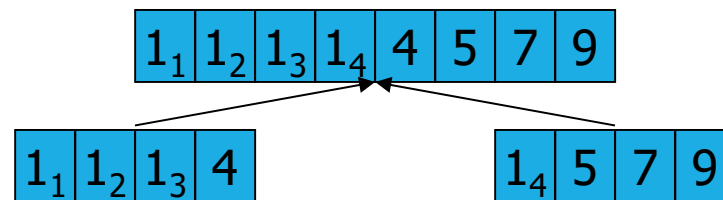
esaurito sottovettore DX

ITEMlt(A[i], A[j]) || ITEMeq(A[i], A[j])

è vero o no? mi sono sganciato dall'operatore == in quanto non devo per forza lavorare su dati interi,

Caratteristiche

- Non in loco (usa il vettore ausiliario B la cui dimensione è funzione di N)
- **Stabile**: in quanto la funzione merge prende dal sottovettore SX in caso di chiavi uguali:



Analisi di complessità

Ipotesi: $n = 2^k$ solo ai fini dell'analisi.

- Dividi: calcola la metà di un vettore $D(n) = \Theta(1)$
- Risolvi: risolve 2 sottoproblemi di dimensione $n/2$ ciascuno $2T(n/2)$
- Terminazione: semplice test $\Theta(1)$
- Combina: basata su Merge $C(n) = \Theta(n)$
- $C(n) + D(n) = \Theta(n)$

Equazione alle ricorrenze:

$$T(n) = 2T(n/2) + n \quad n > 1$$

$$T(1) = 1 \quad n = 1$$

- soluzione per sviluppo (unfolding)

$$T(n/2) = 2T(n/4) + n/2$$

$$T(n/4) = 2T(n/8) + n/4$$

etc.

Terminazione: a ogni passo i dati si dimezzano, dopo i passi sono $n/2^i$. Si termina per $n/2^i = 1$, $i = \log_2 n$

$$T(n) = n + 2*(n/2) + 2^2*(n/4) + 2^3*T(n/8)$$

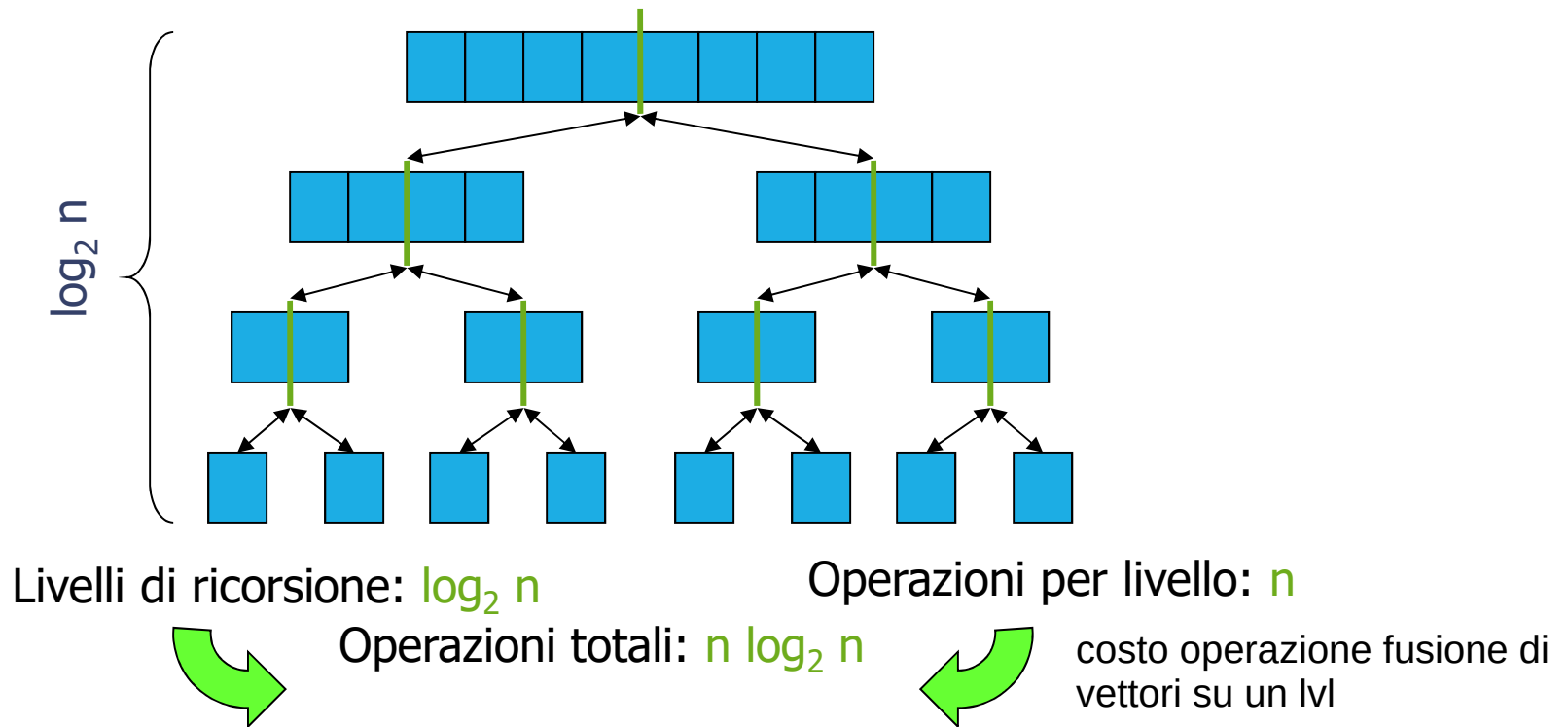
$$= \sum_{0 \leq i \leq \log_2 n} 2^i / 2^i * n = n * \sum_{0 \leq i \leq \log_2 n} 1$$

$$= n*(1 + \log_2 n) = n \log_2 n + n$$

$T(n) = O(n \log n)$. Altri metodi di risoluzione dell'equazione alle ricorrenze portano a $T(n) = \Theta(n \log n)$. la base del logaritmo non conta

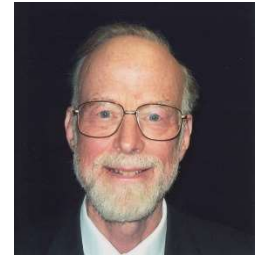
Heap sort ci porta intuitivamente alla complessità $n \log(n)$ quando invece la complessità è lineare

Intuitivamente:



implementazione di quicksort in Qsort() funzione in C

Quicksort (Hoare, 1961)



Divisione:

- partiziona il vettore $A[l..r]$ in due sottovettori SX e DX:
 - dato un elemento pivot $x = A[q]$
 - SX $A[l..q-1]$ contiene tutti elementi $\leq x$
 - DX $A[q+1..r]$ contiene tutti elementi $\geq x$
 - $A[q]$ si trova al posto giusto

La divisione non è necessariamente a metà, a differenza del mergesort.

dove divido (merge sort divide rigidamente in due)
loco (merge sort non in loco)

quick sort basato sul partizionamento:

immagina di prendere un elemento attorno al quale ruota tutto (lo chiamo pivot) e riorganizzo il vettore in due sottovettori, sx con ele minori del pivot e destro quelli maggiori

QUINDI FINITO IL PARTIZIONAMENTO il pivot è già nella SUA POSIZIONE FINALE

Ricorsione

- quicksort su sottovettore SX $A[l..q-1]$
- quicksort su sottovettore DX $A[q+1..r]$
- condizione di terminazione: se il vettore ha 1 elemento è ordinato

Ricombinazione:

- nulla.

mettendo subito tutti i pivot a posto sin da subito NON
devo ricombinare nulla

Partition (à la Hoare)

- Pivot $x = A[r]$
- i per scandire da SX a DX, j da DX a SX
- ripeti fintanto che $i < j$
 - individua $A[i]$ e $A[j]$ elementi “fuori posto”
 - ciclo ascendente su i fino a trovare un elemento maggiore del pivot x
 - ciclo discendente j fino a trovare un elemento minore del pivot x
 - scambia $A[i]$ e $A[j]$
- alla fine scambia $A[i]$ e il pivot x
- ritorna $q = i$
- $T(n) = \Theta(n)$.

PS: esiste anche la partition à la Lomuto.

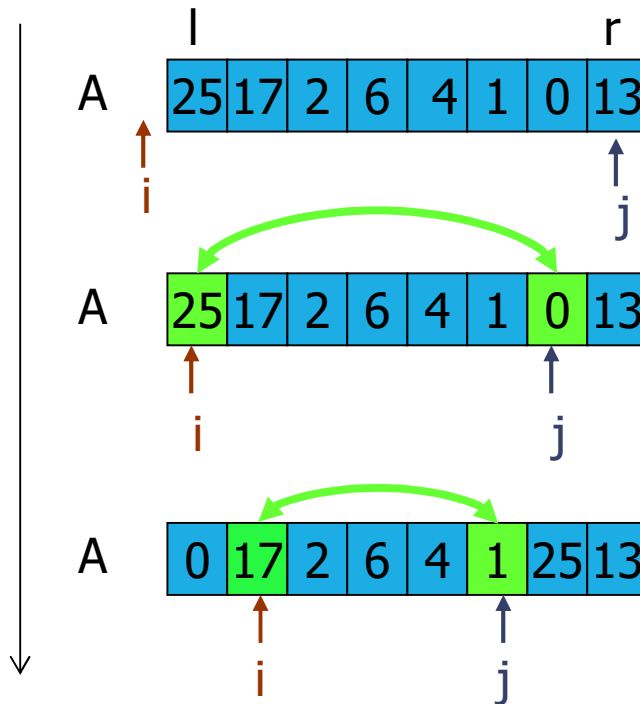
ordino un ele alla volta: ORDINO IL PIVOT

Esempio

indice i vale -1!
in C is not daijoubu

lo scelgo in maniera totalmente arbitraria, x 13
prendiamo l'ultimo ele a caso

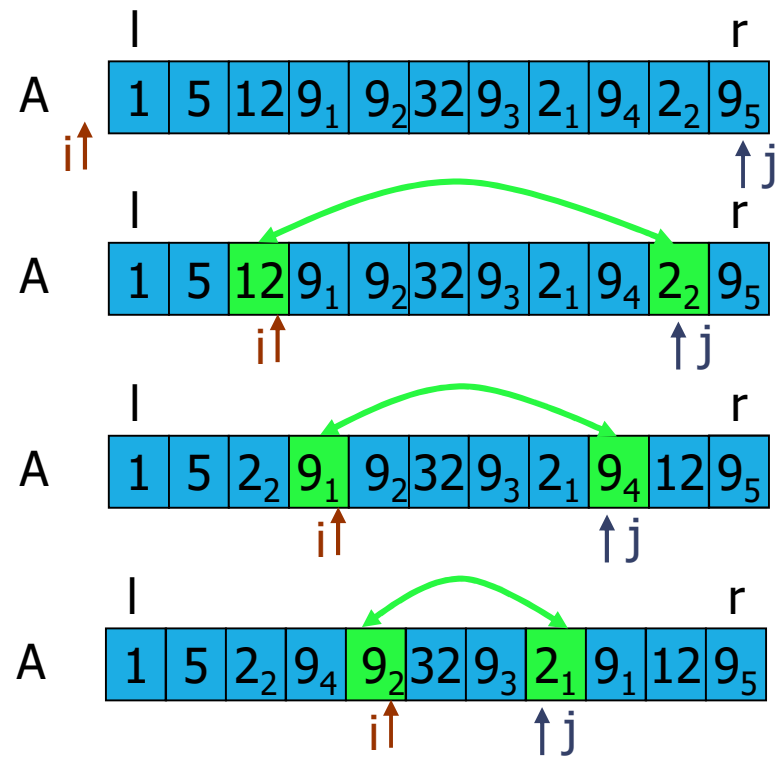
dx con ele
più piccolo
del pivot: sei
dalla parte
sbagliata
lo stesso a
sx (ele +
grande)



il pivot stesso è già nella posizione finale ordinata

Si osservi che, se il pivot compare in più istanze, è irrilevante che esse si trovino alla fine della partition nel sottovettore sinistro o in quello destro, l'importante è che l'istanza scelta come pivot finisca nella posizione finale.

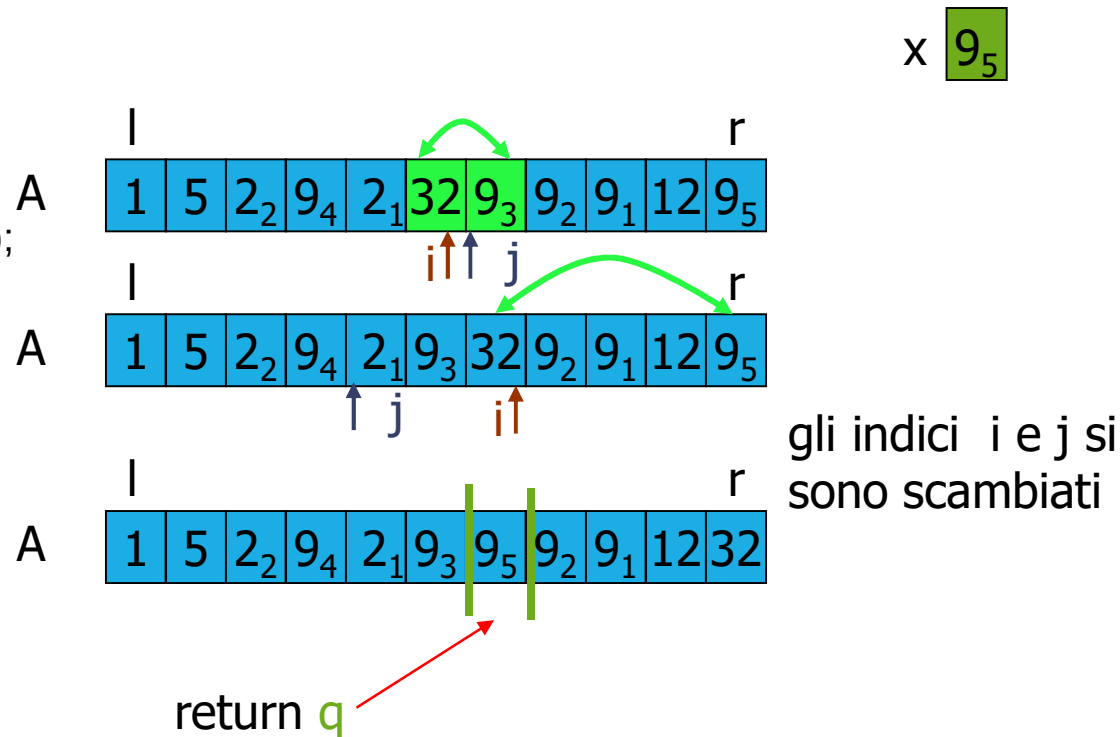
x 9₅



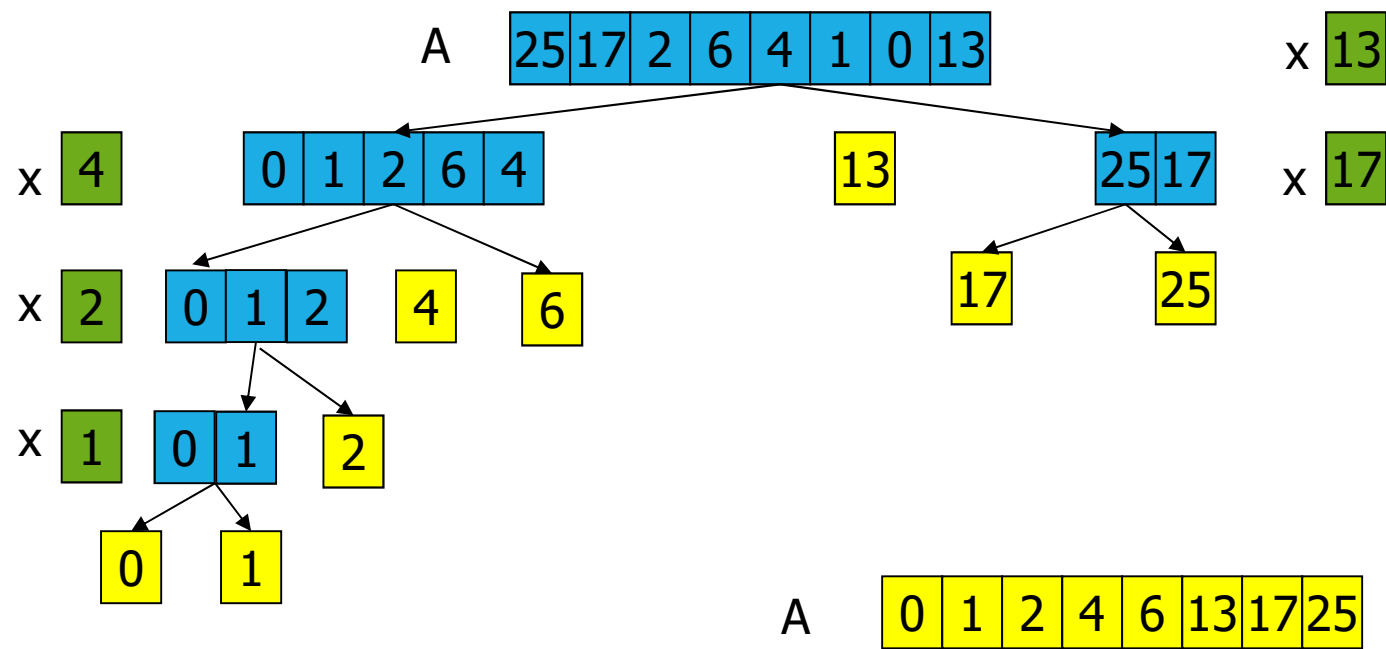
```
// Partitioning algorithm
int partition(int *L, int left, int right){
    int pivot = L[0];
```

```
    while (right > left) {
        while (L[left] < pivot) {
            left = left + 1;
        }
        while (L[right] > pivot) {
            right = right - 1;
        }
        swap(&L[left], &L[right]);
    }
    swap(&pivot, &L[left]);
    return left;
```

//versione comprensibile



Esempio



```

void quickSort(Item A[], int N) {
    int l=0, r=N-1;
    quicksortR(A, l, r);
}

void quicksortR(Item A[], int l, int r ) {
    int q;
    if (l >= r)
        return;
    q = partition(A, l, r);
    quicksortR(A, l, q-1);
    quicksortR(A, q+1, r);
    return;
}

```

wrapper

estremi

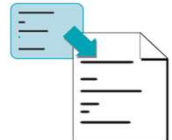
chiamata alla
funzione ricorsiva

terminazione

divisione

chiamata ricorsiva

chiamata ricorsiva



RecursiveSort.c

```

int partition (Item A[], int l, int r ){
    int i = l-1, j = r;  sarebbe ciclo infinito
    Item x = A[r];      es di programmaz. NON
    for ( ; ; ) {       strutturata: esco con break
        while(ITEMlt(A[++i], x)); PRIMA
        while(ITEMgt(A[--j], x)); incremento e
        if (i >= j)      poi lo uso per
            break;       evitare il
        Swap(A, i, j);   problema
    }                   iniziale
    Swap(A, i, r);       dell'indice
    return i;            negativo
}

```

```

void Swap(Item *v, int n1,
          int n2) {
    Item temp;
    temp = v[n1];
    v[n1] = v[n2];
    v[n2] = temp;
    return;
}

```

Caratteristiche

- In loco
- Non stabile: la funzione partition può provocare uno scambio tra elementi «lontani», facendo sì che un'occorrenza di una chiave duplicata si sposti a SX di un'occorrenza precedente della stessa chiave «scavalcandola».

Analisi di complessità

Efficienza legata al bilanciamento delle partizioni

A ogni passo partition ritorna:

- caso peggiore: un vettore da $n-1$ elementi e l'altro da 1 elemento
- caso migliore: due vettori da $n/2$ elementi
- caso medio: due vettori di dimensioni diverse.

Bilanciamento legato alla scelta del pivot.

Caso peggiore

se: vettore ordinato crescente lo voglio decrescente
scegliendo l'ultimo elemento genero da una parte un vettore con
n-1 dati e dall'altro con un elemento

Caso peggiore: pivot = minimo o massimo in vettore già ordinato
in ordine opposto a quello desiderato

in generale se parto dall'ordinamento opposto a
quello cercato è il caso peggiore

Equazione alle ricorrenze:

$$T(n) = T(n-1) + T(1) + n = T(n-1) + n \quad n \geq 2$$

$$T(1) = 1$$

anche se parti da un vettore così puoi prima permutare il vettore
per non avere costo quadratico

Risoluzione per sviluppo:

$$T(n) = n + (n-1) + (n-2) + \dots + 3 + 2 + 1 = \sum_{1 \leq i \leq n} i = n * (n+1)/2$$

Quindi $T(n) = O(n^2)$. Altri metodi di risoluzione dell'equazione alle
ricorrenze portano a $T(n) = \Theta(n^2)$.

Caso migliore

non verrà cagato di striscio...

Equazione alle ricorrenze:

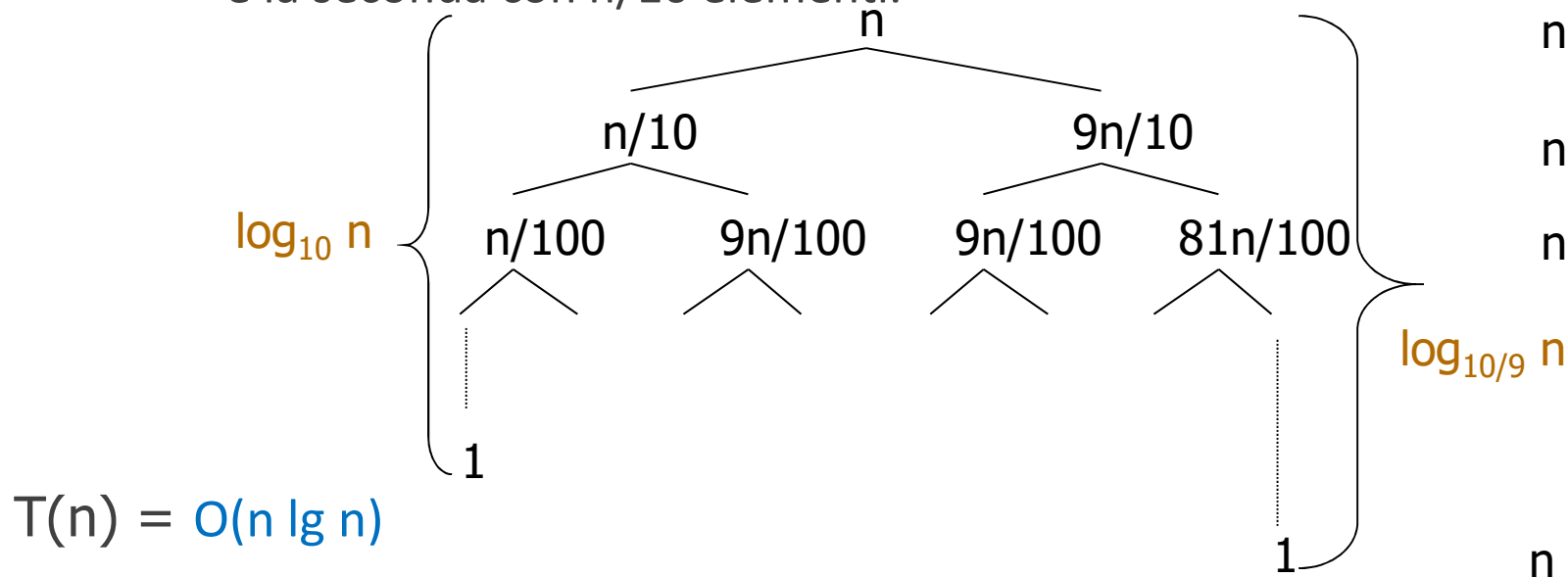
$$T(n) = 2T(n/2) + n \quad n \geq 2$$

$$T(1) = 1$$

$$T(n) = O(n \lg n)$$

Caso medio

- Purché non si ricada nel caso peggiore, anche se il partizionamento è molto sbilanciato, caso medio = caso migliore
- Esempio: ad ogni passo si generano 2 partizioni, la prima con $9/10 n$ e la seconda con $n/10$ elementi.



Scelta del pivot

sceglie uno qualunque, basta che non vada nel caso peggiore

- Elemento di mezzo: $x \leftarrow A[(l+r)/2]$
- Scegliere il valore medio tra min e max
- Scegliere la mediana tra 3 elementi presi a caso nel vettore
- ...

Se lo scopo è semplicemente di rendere molto improbabile il caso peggiore, basta generare un numero casuale i con $l \leq i \leq r$, poi scambiare $A[r]$ e $A[i]$ e infine usare come pivot $A[r]$.

Quadro riassuntivo

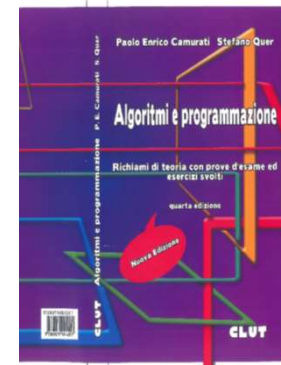
ALGORITMO	IN LOCO	STABILE	COMPLESSITÀ
Bubble Sort	Sì	Sì	$O(n^2)$
Selection Sort	Sì	No	$O(n^2)$
Insertion Sort	Sì	Sì	$O(n^2)$
Shell Sort	Sì	No	dipende
Merge Sort	No	Sì	$O(n \log n)$
Quick Sort	Sì	No	$O(n^2)$
Counting Sort	No	Sì	$O(n)$
Radix Sort	No	Sì	$O(n)$

Riferimenti

- Mergesort
 - Sedgewick 8.3 e 8.5
 - Cormen 2.3
- Quicksort
 - Sedgewick 7.1 e 7.2
 - Cormen 7.1, 7.2

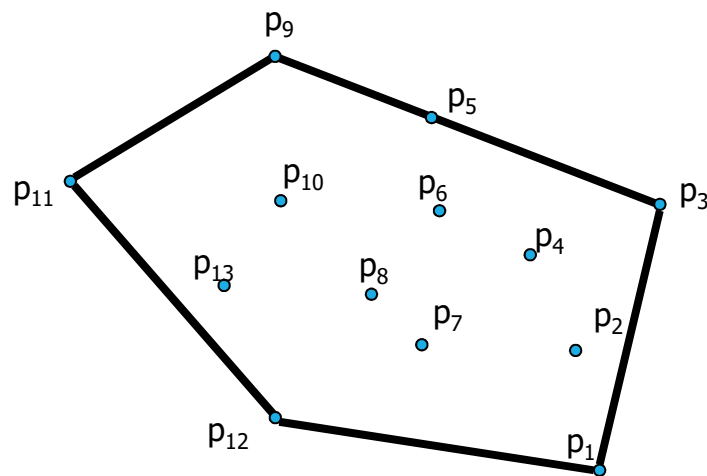
Esercizi di teoria

- 2. Ordinamenti ricorsivi
 - 2.1 Merge Sort
 - 2.2 Quick Sort



Un'applicazione: l'inviluppo convesso

Dato un insieme di punti Q , l'**inviluppo convesso** è il poligono convesso P di area minima per cui ogni punto di Q è interno a P o al più sul perimetro di P :



Soluzione brute-force:

- dato l'insieme dei punti, costruire l'insieme dei suoi sottoinsiemi (insieme delle parti)
- per ciascun sottoinsieme verificare che si tratti di un poligono convesso e se sì calcolarne l'area
- tenere traccia dell'area minima
- complessità esponenziale

Soluzione efficiente: Graham Scan (1972)

- ordinare i punti con Mergesort
- scansione lineare dei punti
- complessità $T(N) = O(N \lg N)$

Il Graham Scan è trattato nei lucidi di approfondimento disponibili sul Portale della Didattica.